Task-6

Task-6 Seaborn For Advanced Civil Engineering Analytics :¶

In [1]:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

In [2]:

```
sns.set_style("whitegrid")
sns.set_palette("deep")
```

In [3]:

```
data = {
    "Concretestrength":[28,28,32,29,31,36,31,30,34,32],
    "WaterCementRatio":
[0.45,0.50,0.55,0.52,0.48,0.46,0.51,0.49,0.53,0.47],
    "CuringDays": [7,14,21,28,28,21,14,7,28,21],
    "LoadCapacity":[80,85,88,92,95,90,87,82,96,91]
}
df = pd.DataFrame(data)
```

1. Histogram¶

In [4]:

```
sns.histplot(df["Concretestrength"])
plt.title("Concrete Strength Distribution")
plt.xlabel("Strength(MPa)")
plt.show()
```

No description has been provided for this image

In [5]:

```
sns.histplot(df["Concretestrength"],kde=True, bins=6)
plt.title("Concrete Strength Distribution")
plt.xlabel("Strength(MPa)")
plt.show()
```

No description has been provided for this image

2- Box Plot :¶

In [6]:

```
sns.boxplot(y=df["Concretestrength"])
plt.title("Box plot of Concrete Strength")
plt.xlabel("Strength(MPa)")
plt.show()
```

No description has been provided for this image

In [7]:

```
sns.boxplot(y=df["Concretestrength"],width=0.4)
plt.title("Box plot of Concrete Strength")
plt.xlabel("Strength(MPa)")
plt.grid(axis='y',linestyle='--',alpha=0.5)
plt.show()
```

No description has been provided for this image

3. Scatter Plot :¶

In [16]:

```
sns.scatterplot(x="WaterCementRatio",y="Concretestrength",data=df)
plt.title("Water-Cement Ratio vs Strength")
plt.xlabel("Water-Cement Ratio")
plt.ylabel("Strength(MPa)")
plt.show()
```

No description has been provided for this image

In [13]:

```
sns.scatterplot(x="WaterCementRatio",y="Concretestrength",data=df,
s=120,marker="o")
plt.title("Water-Cement Ratio vs Strength")
plt.xlabel("Water-Cement Ratio")
plt.ylabel("Strength(MPa)")
plt.show()
```

No description has been provided for this image

4- Bubble Plot :¶

In [17]:

```
sns.scatterplot(x="Concretestrength",y="LoadCapacity",size="CuringDays
",sizes=(50,500),data=df)
plt.title("Bubble Plot: Strength vs Load Capacity")
plt.show()
```

No description has been provided for this image

5. Heat Map :¶

In [18]:

```
corr = df.corr()
sns.heatmap(corr, annot= True)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

No description has been provided for this image

In [19]:

```
corr = df.corr()
sns.heatmap(corr, annot= True,cmap="coolwarm",linewidth=1.2)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

No description has been provided for this image

6- Pairplot :¶

In [21]:

```
sns.pairplot(df)
plt.suptitle("Pairplot of Concrete Strength Dataset", y=1.02)
plt.show()
```

No description has been provided for this image

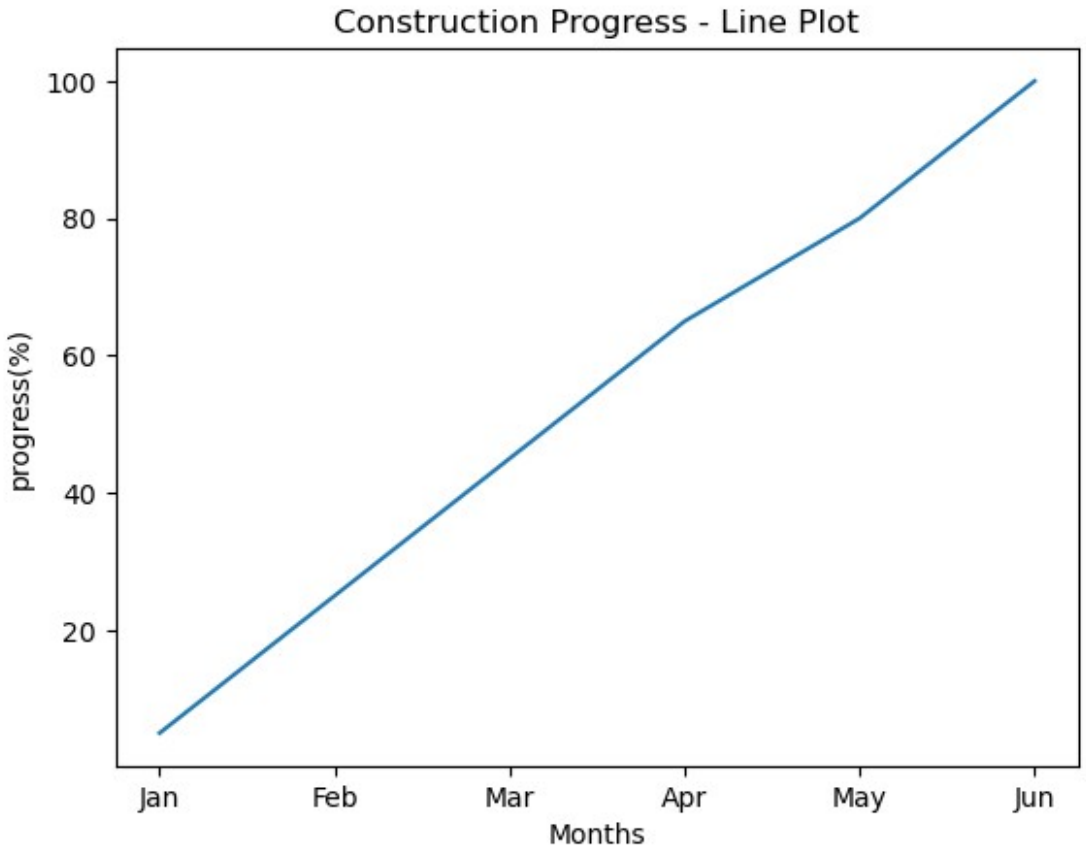
In [ ]:

Task -5 : Matplotlib for Project Visualization

```
import matplotlib.pyplot as plt
import numpy as np
months = [ "Jan","Feb","Mar","Apr","May","Jun"]
progress = [5,25,45,65,80,100]
```

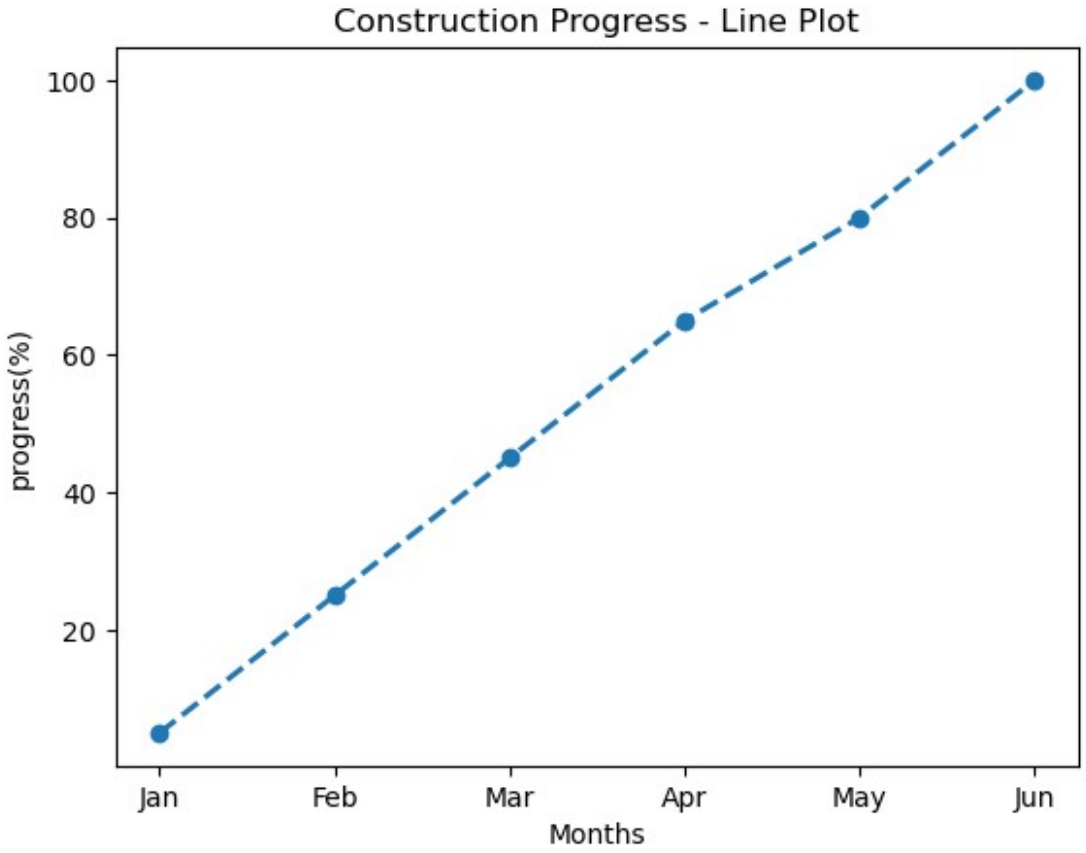
1.Line Plot :

```
plt.plot(months,progress)
plt.title("Construction Progress - Line Plot")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```

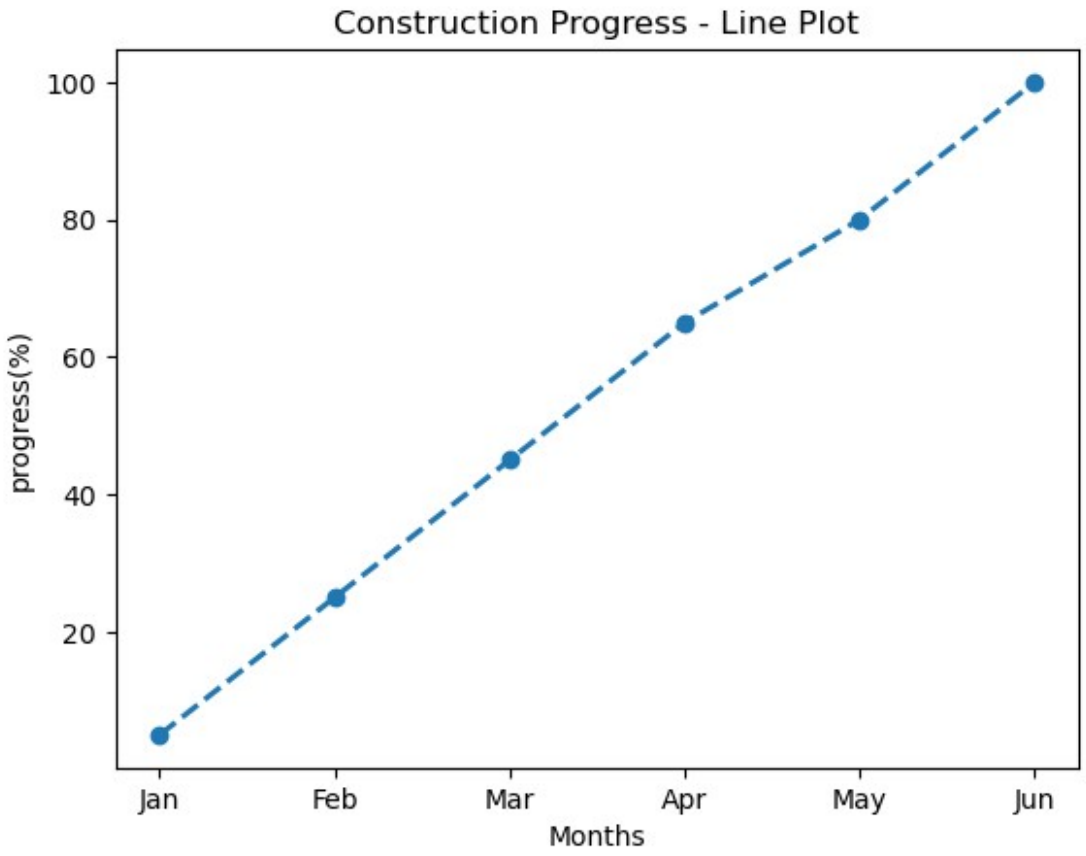



```
plt.plot(months,progress,marker='o',linestyle ='--',linewidth = 2)
plt.title("Construction Progress - Line Plot")
plt.xlabel("Months")
```

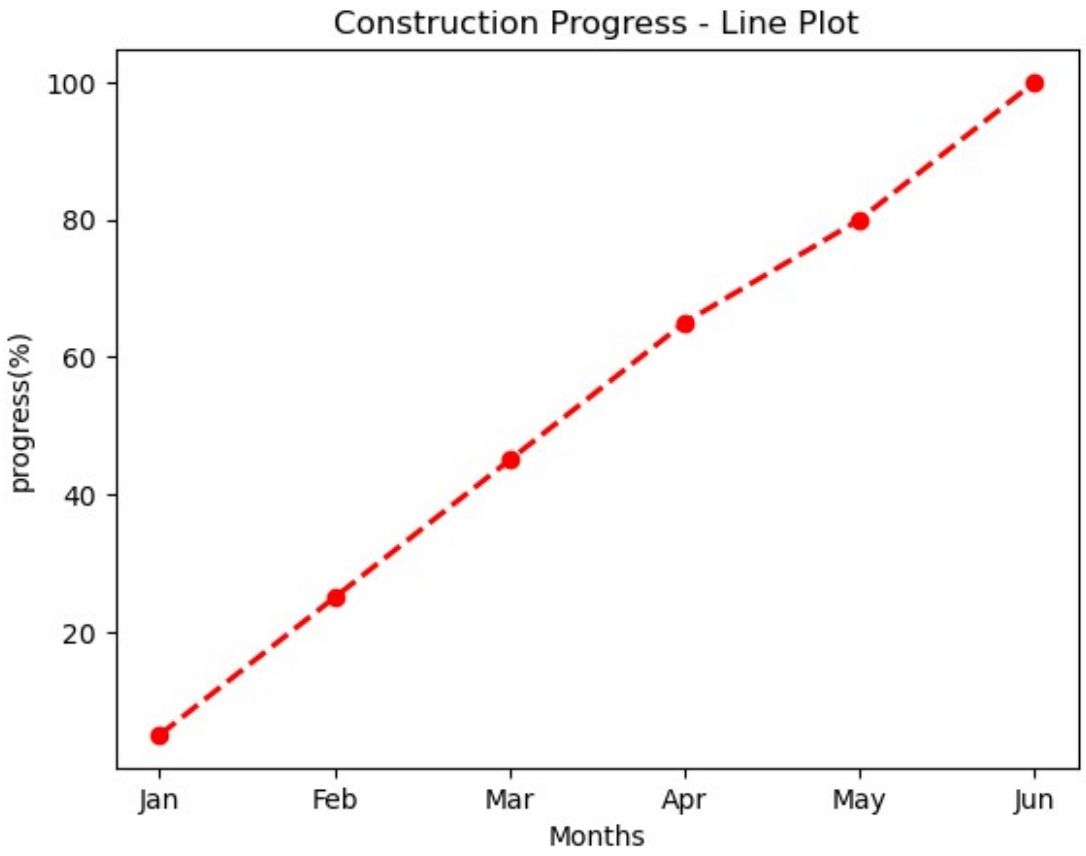
```
plt.ylabel("progress(%)")
plt.show()
```



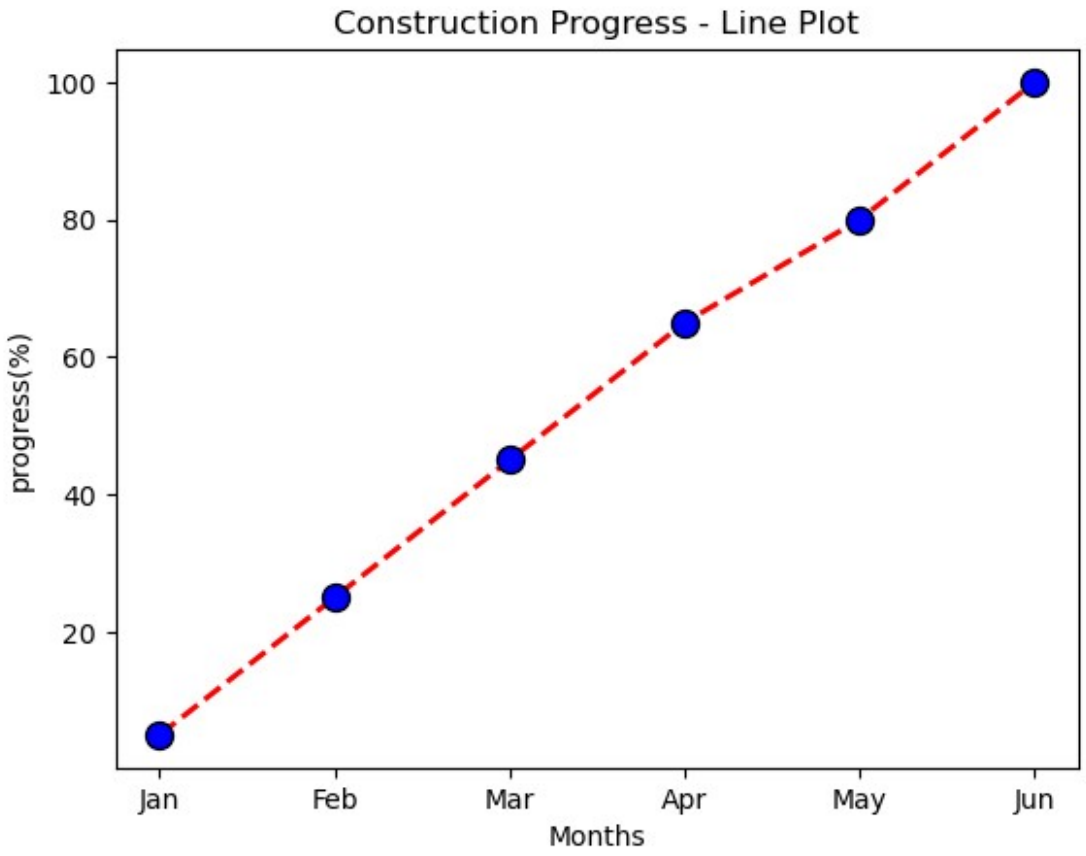

```
plt.plot(months,progress,marker='o',linestyle ='--',linewidth = 2)
plt.title("Construction Progress - Line Plot")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```




```
plt.plot(months,progress,marker='o',linestyle ='--',linewidth =
2,color='red')
plt.title("Construction Progress - Line Plot")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```

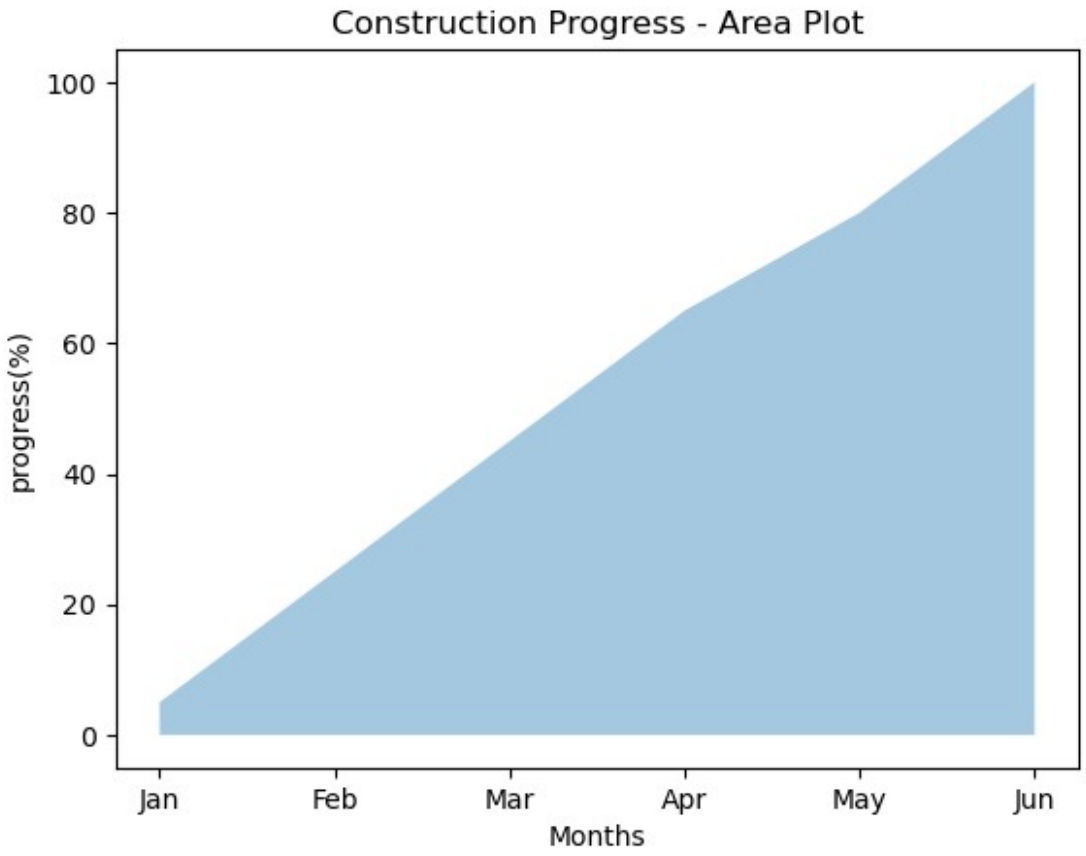



```
plt.plot(months,progress,marker='o',linestyle ='--',linewidth =
2,color='red',markersize=10,markerfacecolor='blue',markeredgecolor='bl
ack')
plt.title("Construction Progress - Line Plot")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```

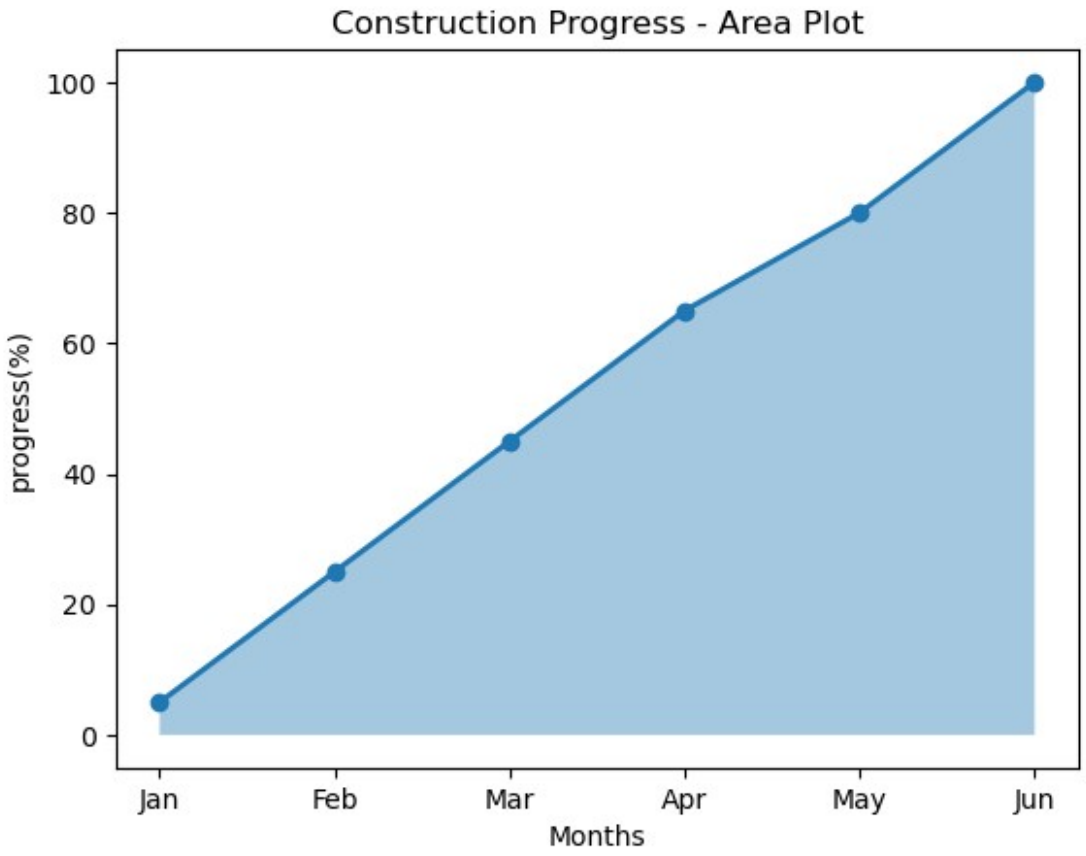


2- Area Plot :

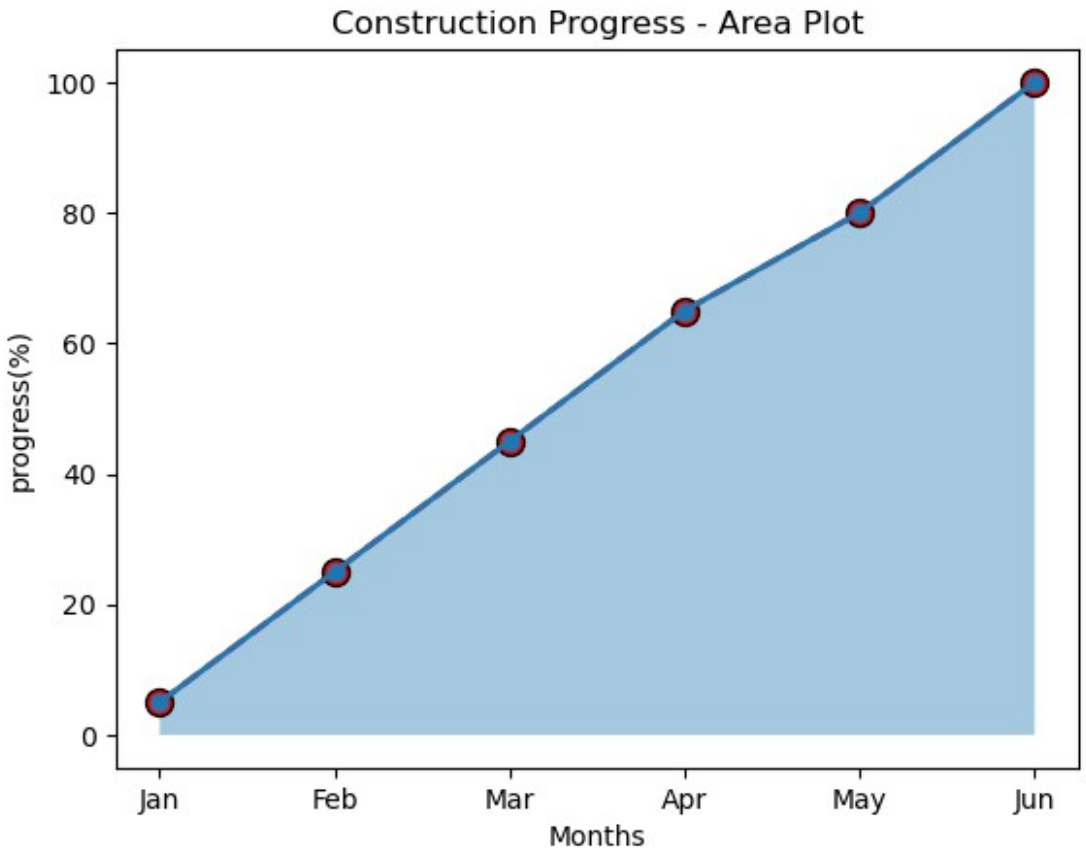
```
plt.fill_between(months,progress,alpha=0.4)
plt.title("Construction Progress - Area Plot")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```




```
plt.fill_between(months,progress,alpha=0.4)
plt.plot(months,progress,marker='o',linewidth=2)
plt.title("Construction Progress - Area Plot")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```

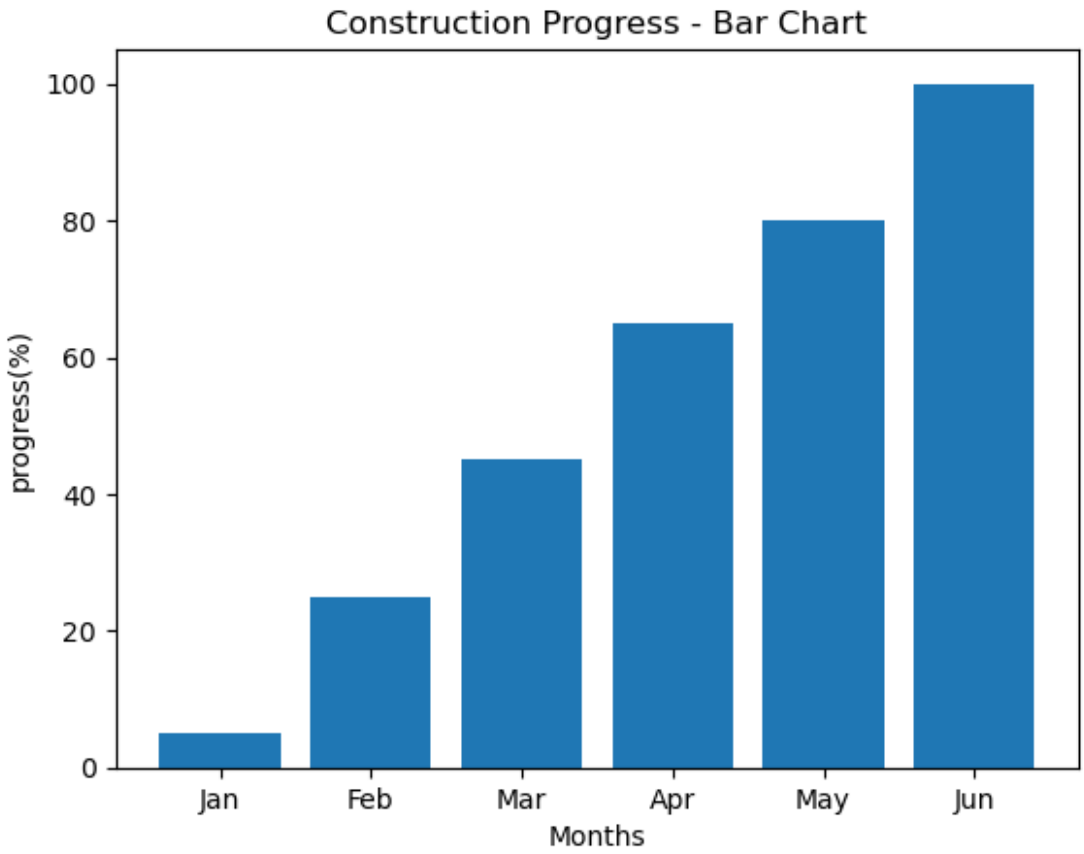



```
plt.fill_between(months,progress,alpha=0.4)
plt.plot(months,progress,marker='o',linestyle ='--',linewidth =
2,color='red',markersize=10,markerfacecolor='red',markeredgecolor='bla
ck')
plt.plot(months,progress,marker='o',linewidth=2)
plt.title("Construction Progress - Area Plot")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```

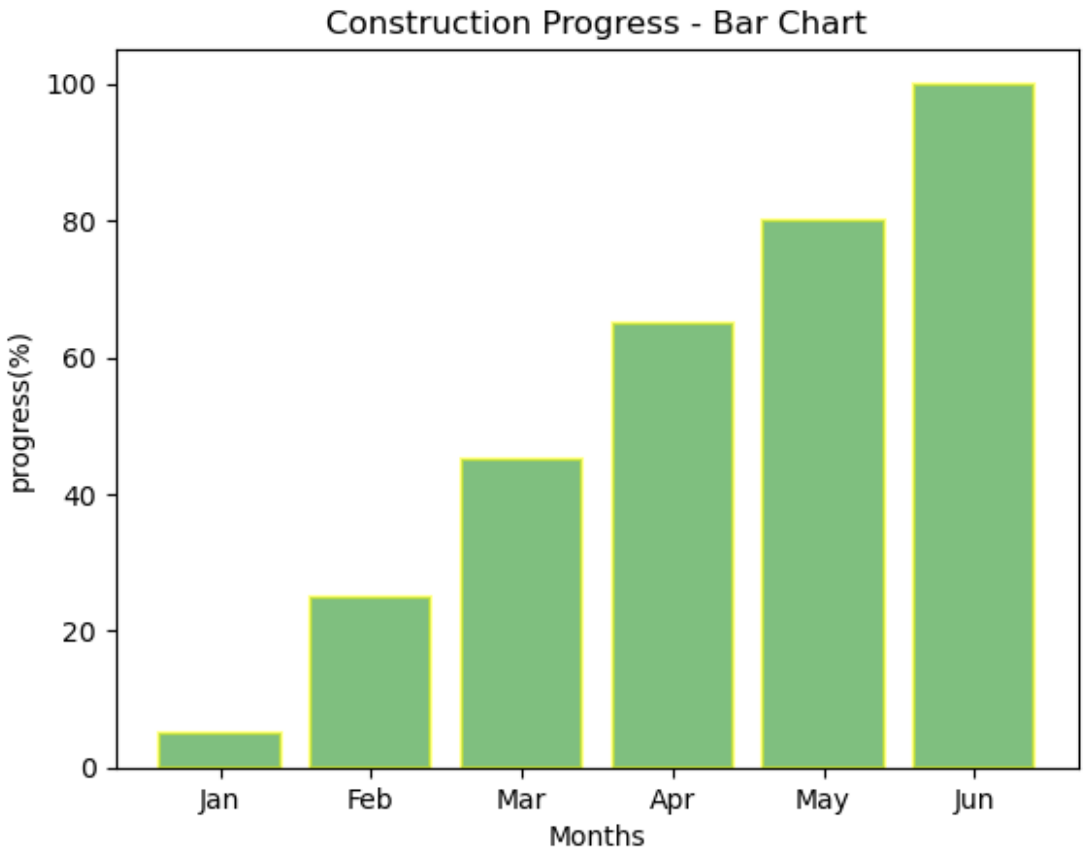


3- Bar Chart

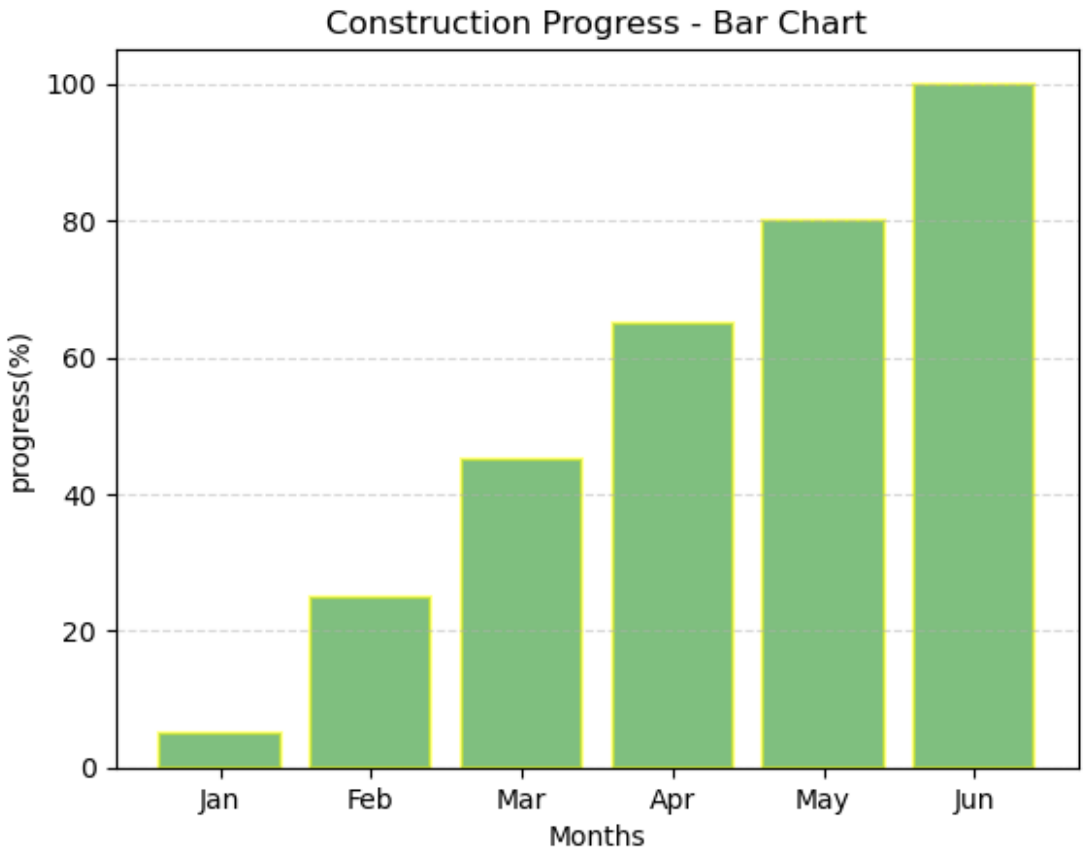
```
plt.bar(months,progress)
plt.title("Construction Progress - Bar Chart")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```



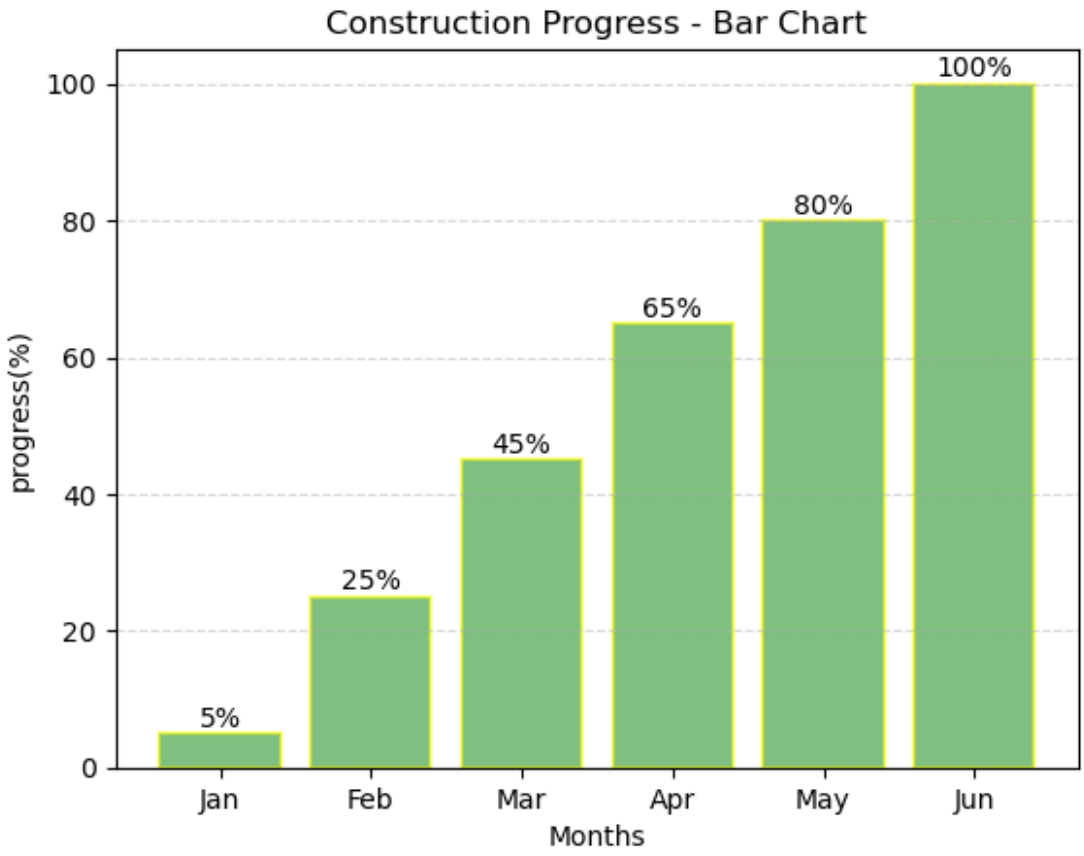

```
plt.bar(months,progress,alpha=0.5,color='Green',edgecolor='yellow',lin
ewidth=1.5)
plt.title("Construction Progress - Bar Chart")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.show()
```




```
plt.bar(months,progress,alpha=0.5,color='Green',edgecolor='yellow',lin
ewidth=1.5)
plt.title("Construction Progress - Bar Chart")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.grid(axis='y',linestyle='--',alpha=0.5)
plt.show()
```

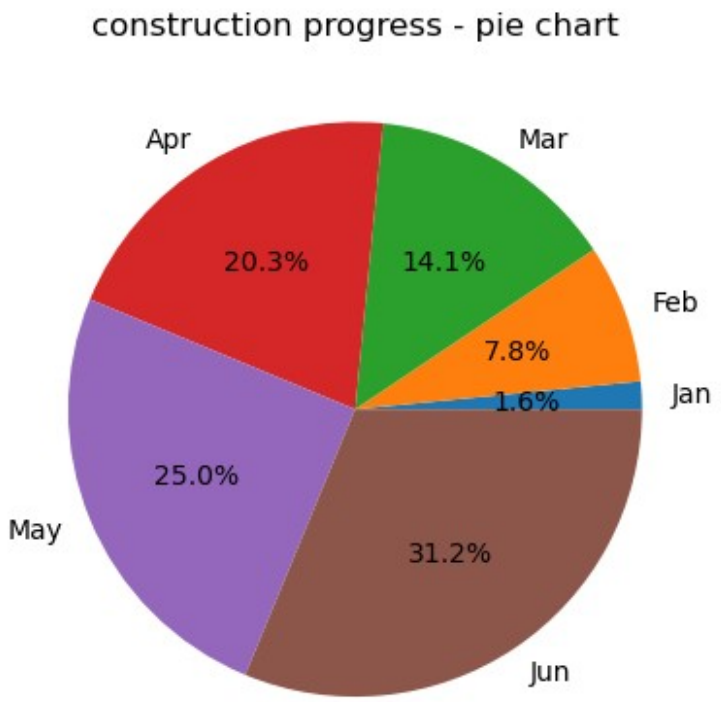



```
plt.bar(months,progress,alpha=0.5,color='Green',edgecolor='yellow',lin
ewidth=1.5)
for i, value in enumerate(progress):
    plt.text(i, value + 1,
             f"{value}%",
             ha='center',
             fontsize=10)
plt.title("Construction Progress - Bar Chart")
plt.xlabel("Months")
plt.ylabel("progress(%)")
plt.grid(axis='y',linestyle='--',alpha=0.5)
plt.show()
```

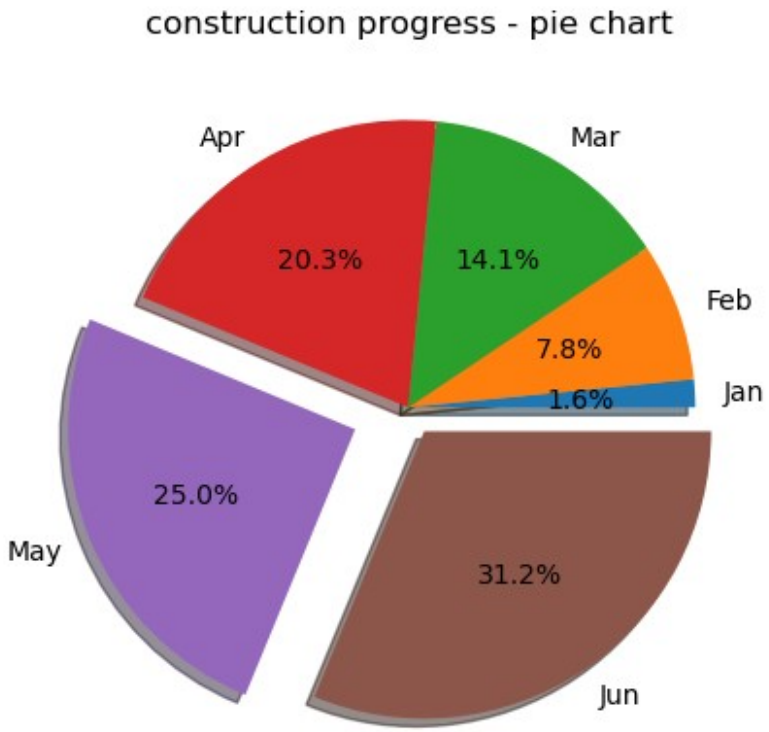


4- Pie Chart:

```
plt.pie(progress, labels=months, autopct="%1.1f%%")
plt.title("construction progress - pie chart")
plt.show()
```




```
explode = [0,0,0,0,0.2,0.1]
plt.pie(progress, labels=months, autopct="%1.1f%%",
explode=explode, shadow = True)
plt.title("construction progress - pie chart")
plt.show()
```


construction progress - pie chart
Apr
Mar
20.3%
14.1%
Feb
7.8%
Jan
1.6%
25.0%
May
31.2%
Jun

Task 4: Pandas for Data Handaling & Cleaning

```
import pandas as pd
```

Series

```
strength_series = pd.Series([25,28,30,32])
print(strength_series)
```

```
0    25
1    28
2    30
3    32
dtype: int64
```

DataFrame :

```
data = {
    "Material":["Concerete","Steel","Brick"],
    "Density(kg/m3)":[2400,7850,1800]
}
materials_df = pd.DataFrame(data)
print("\nMaterial Dataframe : \n", materials_df)
```

```
Material Dataframe :
     Material  Density(kg/m3)
0  Concerete            2400
1      Steel            7850
2      Brick            1800
```

```
schedule = pd.DataFrame({
    "Task": ["Site cleaning","Excavation","Rebar Work","Concereting"],
    "Start Date" : ["2025-01-02","25-01-05","2025-01-10","25-01-15"],
    "End Date" : ["25-01-04","25-01-09","25-01-14","2025-01-18"],
    "Responsible": ["Ravi","Kiran","Meena","Arun"]
})
print(schedule)
```

```
            Task  Start Date    End Date Responsible
0  Site cleaning    25-01-02    25-01-04        Ravi
1     Excavation    25-01-05    25-01-09       Kiran
2     Rebar Work  2025-01-10    25-01-14       Meena
3    Concereting    25-01-15  2025-01-18        Arun
```

Load Data Sheet:

```
df = pd.read_csv("concrete_data.csv")
print(df.head())
```

```
   Cement  Blast Furnace Slag  Fly Ash   Water  Superplasticizer  \
0  287.27              209.45    46.28  187.48              5.23
1  575.36              160.83   135.48  182.29              4.94
2  466.00               92.86   218.24  123.33             18.13
3  399.33              244.14   183.06  164.36              4.99
4  178.01              205.42   201.64  169.43              5.44

   Coarse Aggregate  Fine Aggregate  Age  Strength
0           1131.01          869.08   88     58.55
1           1105.81          918.67  256     78.03
2           1029.41          700.19   73     78.31
3           1182.42          849.95  179     74.22
4            880.19          828.70  212     40.85
```

```
print("\nMissing values  per coloumn:")
print(df.isna().sum())
```

```
Missing values  per coloumn:
Cement                0
Blast Furnace Slag    0
Fly Ash               0
Water                 0
Superplasticizer      0
Coarse Aggregate      0
Fine Aggregate        0
Age                   0
Strength              0
dtype: int64
```

```
print(f"Duplicate Rows : {df.duplicated().sum()}")
```

```
Duplicate Rows : 0
```

```
print("\nData types:")
print(df.dtypes)
print("\nsummary statistics:")
print(df.describe())
```

```
Data types:
Cement                float64
Blast Furnace Slag    float64
Fly Ash               float64
Water                 float64
Superplasticizer      float64
```

```
Coarse Aggregate      float64
Fine Aggregate        float64
Age                     int64
Strength              float64
dtype: object

summary statistics:
           Cement  Blast Furnace Slag     Fly Ash      Water  \
count  500.000000          500.000000  500.000000  500.00000
mean   349.281280          144.585380  129.389680  184.54184
std    149.344305           85.648264   74.298286   37.31138
min    102.530000            1.390000    1.230000  120.42000
25%    220.642500           68.732500   60.305000  151.34250
50%    356.585000          141.545000  134.935000  186.15500
75%    478.065000          217.905000  194.337500  215.86250
max    596.480000          299.920000  249.850000  249.79000

       Superplasticizer  Coarse Aggregate  Fine Aggregate
Age  \
count        500.000000        500.000000      500.000000  500.00000

mean           9.996840       1001.987280      799.135960  187.57000

std            5.716326        118.289252      113.195078  102.36616

min            0.030000        800.000000      600.800000    1.00000

25%            5.357500        903.925000      699.885000   99.75000

50%            9.920000       1000.740000      807.585000  191.00000

75%           14.862500       1107.165000      890.275000  272.25000

max           19.910000       1199.130000      999.560000  364.00000


         Strength
count  500.000000
mean    58.661680
std     17.790489
min      8.930000
25%     45.527500
50%     60.075000
75%     73.135000
max    103.870000
```

Missing Value :

```
data = {
    "Material":["Concrete","Steel","Brick"],
```

```
    "Strength(MPa)":[30,None,10]
}
df = pd.DataFrame(data)
print("Before filling:\n",df)
```

```
Before filling:
    Material  Strength(MPa)
0  Concrete           30.0
1     Steel            NaN
2     Brick           10.0
```

```
df_zero = df.fillna(0)
print("\nAfter filling missing values with 0:\n",df_zero)
```

```
After filling missing values with 0:
    Material  Strength(MPa)
0  Concrete           30.0
1     Steel            0.0
2     Brick           10.0
```

```
df_mean = df.fillna(df.mean(numeric_only=True))
print("\nAfter filling missing values with Mean:\n",df_mean)
```

```
After filling missing values with Mean:
    Material  Strength(MPa)
0  Concrete           30.0
1     Steel           20.0
2     Brick           10.0
```

```
df_dropped = df.dropna()
print("\nAfter droping rows with missing values:\n",df_dropped)
```

```
After droping rows with missing values:
    Material  Strength(MPa)
0  Concrete           30.0
2     Brick           10.0
```

Task -3 : Numpy For Civil Engineering

Foramtion of Array :

```
import numpy as np

bar_diameters = np. array([8,12,16,20])
print("Steel Bar Diameters:",bar_diameters)

Steel Bar Diameters: [ 8 12 16 20]
```

1D Array Properties :

```
print("Dimensions:",bar_diameters.ndim)

Dimensions: 1

print("Shape:",bar_diameters.shape)

Shape: (4,)

print("Data Type:",bar_diameters.dtype)

Data Type: int32

print("Size:",bar_diameters.size)

Size: 4
```

Indexing and Slicing in 1D Array :

```
print("Smallest Diameter:",bar_diameters[0])

Smallest Diameter: 8

print("Diameter from 12mm to 20mm:",bar_diameters[1:])

Diameter from 12mm to 20mm: [12 16 20]
```

2D Array :

```
load_distribution = np.array([
    [10,15,20],
    [12,18,25],
    [14,16,22]
])
print("Load Distribution (kN):\n",load_distribution)
```

```
Load Distribution (kN):
 [[10 15 20]
 [12 18 25]
 [14 16 22]]
```

2D Array properties :

```
print("Dimension:",load_distribution.ndim)
```

```
Dimension: 2
```

```
print("shape:",load_distribution.shape)
```

```
shape: (3, 3)
```

```
print("Data Type:",load_distribution.dtype)
```

```
Data Type: int32
```

3D Array :

```
stress_tensor = np.array([
    [[10,12],[14,16]],
    [[18,20],[22,24]],
    [[26,28],[30,32]],
])
print("Stress Tensor (MPa):\n",stress_tensor)
```

```
Stress Tensor (MPa):
 [[[10 12]
  [14 16]]

 [[18 20]
  [22 24]]

 [[26 28]
  [30 32]]]
```

3D Array Properties

```
print("Dimension:",stress_tensor.ndim)
```

```
Dimension: 3
```

```
print("shape:",stress_tensor.shape)
```

```
shape: (3, 2, 2)
```

```
print("Data Type:",stress_tensor.dtype)
```

```
Data Type: int32
```

Arithemetic Operations :

```
layer1 = np.array([50,60,70])
layer2 = np.array([20,30,40])
total_thickness = layer1 + layer2
print("Total Road Thickness (mm):",total_thickness)
```

```
Total Road Thickness (mm): [ 70  90 110]
```

Statistical Operations :

```
strength = np.array([25,30,28,35,32])
print("Minimum Strength:",np.min(strength))
print("Minimum Strength:",np.max(strength))
print("Mean Strength:",np.mean(strength))
print("Standard Deviation:",np.std(strength))
```

```
Minimum Strength: 25
Minimum Strength: 35
Mean Strength: 30.0
Standard Deviation: 3.40587727318528
```

Task-2 : Python Data Structure for Construction Mnanegement

1.List

```
materials =
["Cement","Steel","Bricks","Sand","Aggregates","Tiles","Paint",]
print(materials)
```

```
['Cement', 'Steel', 'Bricks', 'Sand', 'Aggregates', 'Tiles', 'Paint']
```

```
first_material = materials [0]
last_material = materials [-1]
print(f"First material : {first_material}, Last materials:
{last_material}")
```

```
First material : Cement, Last materials: Paint
```

```
materials[2] = "Cncerete Blocks"
print(materials)
```

```
['Cement', 'Steel', 'Cncerete Blocks', 'Sand', 'Aggregates', 'Tiles',
'Paint']
```

```
materials =
["Cement","Steel","Bricks","Sand","Aggregates","Tiles","Paint",]
Quantity = ["500","200","10000","1500","1200","300","50"]
combined_list = materials + Quantity
print(combined_list)
```

```
['Cement', 'Steel', 'Bricks', 'Sand', 'Aggregates', 'Tiles', 'Paint',
'500', '200', '10000', '1500', '1200', '300', '50']
```

Append :

```
materials.append("Glass")
print(materials)
```

```
['Cement', 'Steel', 'Bricks', 'Sand', 'Aggregates', 'Tiles', 'Paint',
'Glass']
```

Remove :

```
materials.remove("Steel")
print(materials)
```

```
['Cement', 'Bricks', 'Sand', 'Aggregates', 'Tiles', 'Paint', 'Glass']
```

Sort :

```
materials.sort()
print(materials)
```

```
['Aggregates', 'Bricks', 'Cement', 'Glass', 'Paint', 'Sand', 'Tiles']
```

Tuples :

```
cement_tuple = (
    'cement',
    'Binding Material',
    'Ultratech Cement',
    500,
    350,
     2,
     9.5,
     '2024-11-15',
     'N'
)
print(cement_tuple)
```

```
('cement', 'Binding Material', 'Ultratech Cement', 500, 350, 2, 9.5,
'2024-11-15', 'N')
```

```
print('Material Name :', cement_tuple[0])
print('Supplier :', cement_tuple[2])
print('Cost per tonne :', cement_tuple[4])
print('Eco-Friendly?', cement_tuple[-1])
```

```
Material Name : cement
Supplier : Ultratech Cement
Cost per tonne : 350
Eco-Friendly? N
```

Concatenating Tuples :

```
steel_tuples = (
    'steel',
    'Structural Material',
    'Tata Steel',
    300,
    560,
    3,
    9.0,
    '2024-11-10',
    'N'
)
print(steel_tuples)
```

```
('steel', 'Structural Material', 'Tata Steel', 300, 560, 3, 9.0,
'2024-11-10', 'N')
```

Slicing Tuples :

```
key_info = cement_tuple[1:4]
print('Key Information',key_info)
```

```
Key Information ('Binding Material', 'Ultratech Cement', 500)
```

```
key_sort =steel_tuples[1:4]
print('Key Sort',key_sort)
```

```
Key Sort ('Structural Material', 'Tata Steel', 300)
```

Dictionary in Python :

```
construction_materials = {
    "Cement" : 500,
    "Steel" : 70000,
    "Bricks" : 10,
    "Sand" : 1200,
    "Gravel" : 1500,
}
print(construction_materials)
```

```
{'Cement': 500, 'Steel': 70000, 'Bricks': 10, 'Sand': 1200, 'Gravel':
1500}
```

```
print(construction_materials["Cement"])
```

```
500
```

```
print(construction_materials["Steel"])
```

```
70000
```

```
print(construction_materials["Sand"])
```

```
1200
```

Adding materils

```
construction_materials["Concerete"] = 3500
print(construction_materials)
```

```
{'Cement': 500, 'Steel': 70000, 'Bricks': 10, 'Sand': 1200, 'Gravel':
1500, 'Concerete': 3500}
```

Deleting entries

```
del construction_materials["Bricks"]
print(construction_materials)
```

```
{'Cement': 500, 'Steel': 70000, 'Sand': 1200, 'Gravel': 1500,
'Concerete': 3500}
```

checking keys :

```
print("Steel" in  construction_materials)
```

```
True
```

```
print("Wood" in  construction_materials)
```

```
False
```

keys & values :

```
print(construction_materials.keys())
```

```
dict_keys(['Cement', 'Steel', 'Sand', 'Gravel', 'Concerete'])
```

```
print(construction_materials.values())
```

```
dict_values([500, 70000, 1200, 1500, 3500])
```

Nested Dictionary :

```
construction_site = {
    "Floor 1":{"Cement":50,"Steel":100},
    "Floor 2":{"Cement":40,"Steel":1000}
}
print(construction_site)
```

```
{'Floor 1': {'Cement': 50, 'Steel': 100}, 'Floor 2': {'Cement': 40,
'Steel': 1000}}
```

```
print(construction_site["Floor 1"]["Cement"])
```

```
50
```

```
print(construction_site["Floor 2"]["Steel"])
```

```
1000
```

Get() :

```
print(construction_site.get("Floor 2", "Not Available"))
```

```
{'Cement': 40, 'Steel': 1000}
```

```
print(construction_site.get("Floor 3", "Not Available"))
```

```
Not Available
```

try-except block :

```
try:
    print(construction_site["Floor 3"])
except KeyError:
    print("This floor is not available")
```

```
This floor is not available
```

Set :

```
materials = {"Cement","Steel","Concerete","Steel"}
print(materials)
```

```
{'Steel', 'Cement', 'Concerete'}
```

Add in sets :

```
equipment ={"Crane", "Buldozer"}

equipment.add("Excavator")
print("Updated Equipment Set:")
print(equipment)
```

```
Updated Equipment Set:
{'Crane', 'Buldozer', 'Excavator'}
```

Remove In Sets

```
equipment.remove("Excavator")

print("\nUpdated Equipment Set After Removal:")
print(equipment)
```

```


Updated Equipment Set After Removal:
{'Crane', 'Buldozer'}
```

```
materials = {'bricks','Steel','Concerete','Wood'}
print("\n Is 'Steel' in the materials set?")
print('Steel'in materials)
```

```


 Is 'Steel' in the materials set?
True
```

Mathematical Operations With Sets :

Union :

```
foundation_materials = {'steel','concerete','bricks'}
structure_materials = {'steel','wood','concerete'}
all_materials = foundation_materials |  structure_materials
print("All materials ( Union):")
print(all_materials)
```

```
All materials ( Union):
{'bricks', 'steel', 'wood', 'concerete'}
```

Intersection :

```
foundation_materials = {'steel','concerete','bricks'}
structure_materials = {'steel','wood','concerete'}
common_materials = foundation_materials &  structure_materials
print("All materials ( Intersection):")
print(common_materials)
```

```
All materials ( Intersection):
{'steel', 'concerete'}
```

Difference :

```
foundation_materials = {'steel','concerete','bricks'}
structure_materials = {'steel','wood','concerete'}
unique_foundation_materials = foundation_materials -
structure_materials
print("unique_foundation_materials ( Difference):")
print(unique_foundation_materials)
```

```
unique_foundation_materials ( Difference):
{'bricks'}
```

Symmetric Difference :

```
foundation_materials = {'steel','concerete','bricks'}
structure_materials = {'steel','wood','concerete'}
exclusive_materials = foundation_materials ^  structure_materials
print("exclusive_materials ( symmetric difference):")
print(exclusive_materials)
```

```
exclusive_materials ( symmetric difference):
{'wood', 'bricks'}
```

Task 1: Basic Python For Civil Engineering

Data types

1. int

```
num_floors = 5
steel_bars = 120
bricks = 210

print("How many floors?",num_floors)

How many floors? 5

print(" How many steel bars?",steel_bars)

 How many steel bars? 120

print("How many bricks?",bricks)

How many bricks? 210

total_units=num_floors*4
print("How Many Appartment units?",total_units)

How Many Appartment units? 20
```

2. float

```
concrete_volume = 24.5
steel_density = 7.85

print("Volume of concrete:",concrete_volume)

Volume of concrete: 24.5

print("Density of steel density:",steel_density)

Density of steel density: 7.85
```

3. strings

```
project_name = "Skyline Tower"
location = "Chennai"

print("Project Name :",project_name)

Project Name : Skyline Tower
```

```
print("Location:",location)
```

```
Location: Chennai
```

Indexing Examples

print("Character at index 1:",project_name[1]) print("Character at index 4:",project_name[4]) print("Last character:",project_name[-1])

```
project_info = project_name +"_"+ location
print("Project Information:", project_info)
```

```
Project Information: Skyline Tower_Chennai
```

4. Boolean

```
project_on_schedule = True
soil_test_passed = False

print("Is the project on schedule?",project_on_schedule)
print("Did the soil test pass?",soil_test_passed)
```

```
Is the project on schedule? True
Did the soil test pass? False
```

```
type(num_floors)
```

```
int
```

```
type(concrete_volume)
```

```
float
```

```
type(project_name)
```

```
str
```

```
type(project_on_schedule)
```

```
bool
```

Type coversion

```
cost_per_sqft = "5800"
area = 1200

print("What is cost of oer sqft?",cost_per_sqft)
print("How much area is required?",area)
```

```
What is cost of oer sqft? 5800
How much area is required? 1200

type(cost_per_sqft)

str

cost_per_sqft = int(cost_per_sqft)
total_cost = cost_per_sqft*area

print("Total Cost of the building:",total_cost)

Total Cost of the building: 6960000
```

VARIABLES :

```
cement_bags = 20
print("Number of cement bags:",cement_bags)

Number of cement bags: 20
```

Changing Variable Value :

```
cement_bags = 20
cement_bags = 15

print("Updated cement bags:",cement_bags)

Updated cement bags: 15
```

Expressions

```
total_concerete = 100+60
print(f"Total concerete required:{total_concerete} cubic meteres")

Total concerete required:160 cubic meteres
```

Substraction(-)

```
remaining_budget = 10000-15000
print(f"Remaming Budget:{remaining_budget} INR")

Remaming Budget:-5000 INR

cost_per_rod = 5000
num_rods = 5
total_cost = cost_per_rod*num_rods
print(f"Total steel Cost: {total_cost} INR")

Total steel Cost: 25000 INR
```

DIVISION(/):

```
total_budget = 25000
sections = 5
cost_per_section = total_budget/sections
print(f"cost per section : {cost_per_section} INR")
```

```
cost per section : 5000.0 INR
```

Integer Division

```
total_workers = 25
floors = 6
workers_per_floors = total_workers // floors
print(f"Workerss per floors:{workers_per_floors}")
```

```
Workerss per floors:4
```

Paretheses

```
material_cost = 500
tax_rate =1.18
labor_cost = 300
total_project_cost = (material_cost + labor_cost)*tax_rate
print(f" Total project cost : {total_project_cost}")
```

```
 Total project cost : 944.0
```

Comparison Operators :

```
material_strength = 150
required_strength = 200
print(material_strength == required_strength)
```

```
False
```

```
print(material_strength >= required_strength)
```

```
False
```

```
print(material_strength < required_strength)
```

```
True
```

CONDITIONAL STATEMENT IN PYTHON

```
strength = 250
if strength > 200:
    print("proceed with construction.")
```

```
else:
    print("Select a stronger material.")
```

```
proceed with construction.
```

```
strength = 250
if strength > 250:
    print("Use current mix design.")
elif strength > 200:
    print("Print with adjusted mix design.")
else:
    print("Select a stronger material.")
```

```
Print with adjusted mix design.
```

Logical Operators

```
material = "steel"
strenth = 220
load_capacity = 500
suitable = False
if not suitable:
    print("Material is unsuitable.")
```

```
Material is unsuitable.
```

```
if material == "steel" or material == "concerete":
    print("Material is acceptable.")
```

```
Material is acceptable.
```

```
strength = 220
load_capacity = 500
if strength > 200 and load_capacity > 400:
    print("Material is suitable .")
```

```
Material is suitable .
```

LOOP :

```
materials_cost = [1500,2000,2500,1800]
total_cost = 0
for cost in materials_cost:
    total_cost += cost
    print(f"Total material cost:{total_cost} INR")
```

```
Total material cost:1500 INR
Total material cost:3500 INR
Total material cost:6000 INR
Total material cost:7800 INR
```

```
materials_cost = [1500,2000,2500,1800]
total_cost = 0
i = 0
while i < len(materials_cost):
    total_cost += materials_cost[i]
    i+=1
    print(f"Total material cost:{total_cost} INR")
```

```
Total material cost:1500 INR
Total material cost:3500 INR
Total material cost:6000 INR
Total material cost:7800 INR
```

Break and Continue :

```
materials_cost = [1500,-1200,2000,2500,-800]
total_cost = 0
for cost in materials_cost:
    if cost < 0:
        print(f"Invalid cost ({cost}) skipped.")
        continue
        total_cost += cost
        print(f"Total Valid material cost:{total_cost} INR")
```

```
Invalid cost (-1200) skipped.
Invalid cost (-800) skipped.
```

break :

```
materials_cost = [1500,2000,2800,3200,2500]
total_cost = 0
for cost in materials_cost:
    if cost < 3000:
        print(f"Material cost is too high,stopping calculation.")
        break
        total_cost += cost
        print(f"Total Valid material cost:{total_cost} INR")
```

```
Material cost is too high,stopping calculation.
```